

Teste Técnico — Analytics Engineer

Tempo sugerido: 3 a 4 horas (não cronometrado) **Entregáveis:** repositório Git + documento de decisões (README)

Contexto

Você acaba de entrar na **FinLend** e herdou um projeto dbt que foi construído às pressas. Os analistas reclamam que "os números nunca batem", o time de produto quer começar a usar IA para responder perguntas de negócio via linguagem natural, e o CFO acabou de questionar por que a conta do BigQuery triplicou no último trimestre.

Abaixo está o estado atual do projeto. Seu trabalho começa aqui.

O que você recebeu

Repositório legado

O projeto dbt atual tem a seguinte estrutura:

```
models/  
├── staging/  
│   └── stg_transactions.sql  
├── marts/  
│   ├── revenue_report.sql  
│   └── merchant_summary.sql  
└── schema.yml
```

models/staging/stg_transactions.sql

```
-- Último autor: estagiário que saiu em janeiro  
-- "funciona, não mexe"
```

```
SELECT  
    transaction_id,  
    merchant_id,  
    customer_id,  
    amount_cents,  
    amount_cents / 100 as amount_brl,  
    status,  
    payment_method,
```

```

    created_at,
    updated_at,
    metadata,
    CURRENT_TIMESTAMP() as loaded_at
FROM {{ source('raw', 'transactions') }}
WHERE status != 'test'

```

models/marts/revenue_report.sql

```

{{
    config(
        materialized='table',
        partition_by={
            "field": "transaction_date",
            "data_type": "date",
            "granularity": "day"
        }
    )
}}

WITH base AS (
    SELECT
        t.transaction_id,
        t.merchant_id,
        m.trade_name as merchant_name,
        m.mcc_code,
        t.amount_br1,
        t.status,
        t.payment_method,
        DATE(t.created_at) as transaction_date,
        t.created_at,
        s.settlement_id,
        s.net_amount_cents / 100 as net_amount,
        s.fee_amount_cents / 100 as fee_amount,
        s.settlement_date,
        s.paid_at
    FROM {{ ref('stg_transactions') }} t
    LEFT JOIN {{ source('raw', 'merchants') }} m
        ON t.merchant_id = m.id
    LEFT JOIN {{ source('raw', 'settlements') }} s
        ON t.transaction_id IN UNNEST(s.transaction_ids)
    WHERE t.status IN ('captured', 'refunded', 'chargeback')
)

SELECT
    *,
    CASE
        WHEN status = 'captured' THEN amount_br1
        WHEN status = 'refunded' THEN -amount_br1
        WHEN status = 'chargeback' THEN -amount_br1
    
```

```

    END as revenue_impact,
    ROW_NUMBER() OVER (
        PARTITION BY transaction_id
        ORDER BY settlement_date DESC
    ) as rn
FROM base
QUALIFY rn = 1

```

models/marts/merchant_summary.sql

```
{{ config(materialized='table') }}
```

```

SELECT
    merchant_id,
    merchant_name,
    mcc_code,
    COUNT(*) as total_transactions,
    SUM(revenue_impact) as total_revenue,
    SUM(fee_amount) as total_fees,
    SUM(CASE WHEN status = 'chargeback' THEN 1 ELSE 0 END) as chargebacks,
    SUM(CASE WHEN status = 'chargeback' THEN 1 ELSE 0 END) / COUNT(*) as chargeback_rate,
    MIN(transaction_date) as first_transaction,
    MAX(transaction_date) as last_transaction
FROM {{ ref('revenue_report') }}
GROUP BY 1, 2, 3

```

models/schema.yml

```
version: 2
```

```
sources:
```

- name: raw
- tables:
 - name: transactions
 - name: merchants
 - name: settlements

```
models:
```

- name: stg_transactions
 description: "staging transactions"
- name: revenue_report
 description: "report for revenue"
- name: merchant_summary
 description: "summary by merchant"

O que você deve entregar

Parte 1 — Diagnóstico (obrigatório)

Analise o projeto legado acima e produza um **diagnóstico por escrito** listando tudo que está errado ou abaixo do padrão que você esperaria. Para cada problema, explique:

- O que está errado e por que isso é um problema real (não teórico)
- Qual o impacto no negócio, nos custos ou na confiabilidade dos dados
- Como você corrigiria

Não precisa corrigir tudo em código — queremos ver sua capacidade de leitura crítica e priorização. Ordene os problemas do mais grave ao menos grave.

Parte 2 — Refatoração seletiva (obrigatório)

Escolha os **3 problemas mais críticos** do seu diagnóstico e implemente a correção em código dbt. Entregue o projeto refatorado no repositório Git com justificativa (no README) de por que escolheu esses 3 e não outros

Você pode reorganizar a estrutura de pastas, criar novos modelos, remover o que quiser. O projeto é seu agora.

Parte 3 — Preparação para IA (obrigatório)

O time de produto quer que um agente de IA consiga responder perguntas como:

- `""Qual o volume de transações Pix do merchant X no último mês?""`
- `""Quais merchants tiveram taxa de chargeback acima de 2% esse trimestre?""`
- `""Quanto a Franq faturou em taxas na última semana?""`

Usando o(s) modelo(s) que você refatorou, prepare-os para consumo por IA:

1. Enriqueça o `schema.yml` com `meta` tags, descrições e qualquer metadado que ajude um LLM a gerar SQL correto
2. Em um arquivo `SEMANTIC_NOTES.md`, documente:
 - Quais perguntas de negócio esse modelo consegue responder
 - Quais armadilhas um agente cometeria sem orientação (e como suas `meta` tags previnem isso)
 - O que falta para uma camada semântica completa

Parte 4 — Documento de decisões (obrigatório)

No README, inclua:

1. **Seu processo:** como abordou o problema, por onde começou, como priorizou
2. **Uso de IA:** quais ferramentas usou, para quê, e onde o output da IA estava errado ou insuficiente e você precisou intervir
3. **O que faria com mais tempo:** se tivesse mais tempo, qual seria seu plano de ação priorizado?

Parte 5 — Desafio extra (opcional)

Escolha um:

- **Custo:** a conta do BigQuery triplicou. Sem acesso ao `INFORMATION_SCHEMA`, olhando apenas para o código dbt acima, identifique o que provavelmente está causando o custo elevado e proponha (ou implemente) soluções.
- **Teste de negócio:** implemente um teste singular dbt que valide uma regra de negócio que você identificou como crítica durante o diagnóstico.
- **Orquestração:** esboce como organizaria a execução em produção (Prefect, Airflow, dbt Cloud — à sua escolha). Considere dependências, retries, alertas e frequência.

Observações Finais

- O uso de IA é encorajado. Copiar e colar sem revisar será perceptível.
- Se algo for ambíguo, tome uma decisão, documente e siga em frente. Autonomia é parte da avaliação.
- Valorizamos profundidade de raciocínio mais do que volume de código.
- Não existe resposta certa — existem decisões bem justificadas.